

AL-AZHARI LIBRARY FRONTEND

PHASE 3 — AUTHENTICATION COMPLETION REPORT

A. Executive Summary

Phase 3 (Authentication) of the **AL-AZHARI Library** web frontend has been implemented according to the project specifications and the architecture defined in the implementation plan.

The implementation delivers an authentication foundation adhering to:

- **Zero localStorage/sessionStorage Token Storage:** Access tokens are kept in-memory only via AuthTokenHolder.
- **HttpOnly Refresh Cookies:** Refresh tokens are managed by the backend using HttpOnly cookie `al_azhari_refresh` via credentials-enabled HTTP requests (`withCredentials: true`).
- **Reactive Signals Store:** AuthStore provides reactive signals for session lifecycle (`status` , `user` , `error` , `isPending` , `restoreFailed` , `isAuthenticated` , `isEmailUnverified`).
- **Seamless Bootstrapping & SSR Safety:** Configured `provideAppInitializer` in `app.config.ts` executes `authStore.restore()` during application startup without delaying hydration, safely bypassing on SSR.
- **Defensive Route Protection:** `authGuard` and `guestOnlyGuard` with strict sanitization of `returnUrl` preventing open redirect attacks.
- **RTL-First Accessible Shell & Pages:** Standardized on Phase 1 design tokens, CSS logical properties (`margin-block` , `margin-inline` , `inline-size` , `block-size`), and Phase 1 form primitives.
- **Verification Gates:** TypeScript checks pass with 0 errors; ESLint passes with 0 warnings; Stylelint passes with 0 warnings; Production build completes with 0 warnings; All **166 / 166** unit tests pass.

B. Architecture & Directory Tree

```

frontend/src/app/
├─ app.config.ts          # [MODIFIED] Added provideAppInitializer for AuthStore.restore()
├─ app.routes.ts          # [MODIFIED] Registered lazy auth feature routes
├─ core/
│   └─ api/
│       ├── account/
│       │   ├── auth-api.service.ts    # [NEW] Typed wrapper for /auth and /me endpoints
│       │   └─ auth-api.service.spec.ts # [NEW] Unit tests for AuthApi
│       ├── dto/
│       │   └─ auth.dto.ts             # [NEW] Request/response DTO contracts matching backend
│       └─ mappers/
│           └─ auth.mapper.ts          # [NEW] DTO -> Domain model mapping functions
│   └─ auth/
│       ├── auth.store.ts              # [NEW] Signals-based authentication store
│       ├── auth.store.spec.ts         # [NEW] Unit tests for AuthStore lifecycle
│       ├── guards/
│       │   ├── auth.guard.ts          # [NEW] Route guard + sanitizeReturnUrl()
│       │   ├── auth.guard.spec.ts     # [NEW] Unit tests for authGuard
│       │   ├── guest-only.guard.ts    # [NEW] Route guard for login/register pages
│       │   └─ guest-only.guard.spec.ts # [NEW] Unit tests for guestOnlyGuard
│       ├── token-holder.service.ts    # [Phase 2] In-memory access token repository
│       └─ index.ts                   # [NEW] Barrel exports for auth module
├─ domain/
│   └─ models/
│       └─ auth.model.ts              # [NEW] AuthUser, AuthStatus, UserRole domain models
├─ layout/
│   └─ auth-shell/
│       ├── auth-shell.component.ts    # [NEW] Dedicated standalone layout shell
│       ├── auth-shell.component.scss  # [NEW] Scss with logical properties and RTL support
│       └─ auth-shell.component.spec.ts # [NEW] Unit tests for AuthShellComponent
└─ features/
    └─ auth/
        ├── auth.routes.ts            # [NEW] Child routes under AuthShellComponent
        ├── login/
        │   ├── login.component.ts     # [NEW] Login page with form controls and returnUrl
        │   ├── login.component.scss   # [NEW] Scss for login page
        │   └─ login.component.spec.ts # [NEW] Unit tests for LoginComponent
        ├── register/
        │   ├── register.component.ts   # [NEW] Registration page with validation & password match
        │   ├── register.component.scss # [NEW] Scss for register page
        │   └─ register.component.spec.ts # [NEW] Unit tests for RegisterComponent
        ├── forgot-password/
        │   ├── forgot-password.component.ts # [NEW] Password reset dispatch form
        │   ├── forgot-password.component.scss # [NEW] Scss for forgot-password page
        │   └─ forgot-password.component.spec.ts # [NEW] Unit tests for ForgotPasswordComponent
        ├── reset-password/
        │   ├── reset-password.component.ts # [NEW] Password reset submission form
        │   ├── reset-password.component.scss # [NEW] Scss for reset-password page
        │   └─ reset-password.component.spec.ts # [NEW] Unit tests for ResetPasswordComponent
        └─ verify-email/
            ├── verify-email.component.ts # [NEW] Email verification landing page
            ├── verify-email.component.scss # [NEW] Scss for verify-email page
            └─ verify-email.component.spec.ts # [NEW] Unit tests for VerifyEmailComponent

```

C. Backend API Integration & Contract Verification

The frontend auth implementation communicates with the Fastify/Prisma backend (Back-end/src/modules/auth/):

Endpoint	Method	Payload / Params	Credentials	Response Contract
/auth/login	POST	{ identifier, password }	withCredentials: true	{ accessToken, user } + sets al_azhari_refresh cookie
/auth/register	POST	{ name, email, phone, password }	Default	{ id, name, email, phone, role, status }
/auth/refresh	POST	{ }	withCredentials: true	{ accessToken } + rotates al_azhari_refresh cookie
/auth/logout	POST	{ all?: boolean }	withCredentials: true	204 No Content / 200 + clears cookie
/auth/verify-email	POST	{ token }	Default	{ user, verified: true }
/auth/forgot-password	POST	{ email }	Default	202 Accepted { message: '...' } (anti-enumeration)
/auth/reset-password	POST	{ token, newPassword }	Default	204 No Content
/me	GET	Bearer <accessToken>	Default	{ id, name, email, phone, role, status }

D. Security Specifications

1. Token Hygiene:

- Access tokens are stored exclusively in Javascript heap memory within AuthTokenHolder.
- Neither localStorage nor sessionStorage nor client-accessible cookies contain the access token, neutralizing persistent XSS token theft.
- Refresh tokens are protected by backend HttpOnly , SameSite=Lax , Path=/api/v1/auth cookies.

2. Open-Redirect Hardening:

- The helper function sanitizeReturnUrl() in auth.guard.ts validates that redirect targets begin strictly with a single / and cannot specify external schemes (//evil.com , https://... , javascript:). Non-conforming URLs default to / .

3. Anti-Enumeration Handling:

- The forgot password flow displays a confirmation notice regardless of whether the email exists in the database.

4. Automatic Token Refresh:

- AuthStore registers its refresh routine into AuthTokenHolder via registerRefreshHandler() . When an authenticated API call receives a 401 response, authInterceptor coordinates transparent background refresh and retries the request once before terminating the session.

E. Signal-Based AuthStore Design

AuthStore exposes reactive read-only signals:

- status : 'unknown' | 'anonymous' | 'authenticated' | 'refreshing' | 'expired'
- user : AuthUser | null
- error : string | null (localized user-facing messages)
- isPending : boolean (loading state for auth mutations)
- restoreFailed : boolean (transient network or server failure indicator)
- isAuthenticated : computed(() => status() === 'authenticated')

- `isAnonymous`: `computed(() => status() === 'anonymous')`
- `isEmailUnverified`: `computed(() => Boolean(user && !user.isEmailVerified))`
- `role`: `computed(() => user?.role ?? null)`

F. Routing & Page Architecture

Auth routes are configured under `auth.routes.ts` and mounted in `app.routes.ts`:

Path	Component	Guard	Description
<code>/login</code>	<code>LoginComponent</code>	<code>guestOnlyGuard</code>	Email/phone and password login with return URL routing
<code>/register</code>	<code>RegisterComponent</code>	<code>guestOnlyGuard</code>	Registration with Egyptian mobile validation & password confirmation
<code>/forgot-password</code>	<code>ForgotPasswordComponent</code>	<code>guestOnlyGuard</code>	Password reset link request
<code>/reset-password</code>	<code>ResetPasswordComponent</code>	<code>guestOnlyGuard</code>	Token-based password reset form
<code>/verify-email</code>	<code>VerifyEmailComponent</code>	<code>None</code>	Landing page for email verification links

G. Design System & Accessibility Compliance

- **Design Tokens:** Standardized on CSS variables (`--color-primary-*` , `--color-danger-*` , `--space-*` , `--radius-*`).
- **Logical Properties:** Enforced through Stylelint; all margin, padding, border, and dimensional attributes use logical specs (`margin-block` , `margin-inline` , `inline-size` , `block-size`).
- **RTL-First Typography:** Arabic text layout utilizing Amiri and Cairo font stacks with system Latin fallbacks.
- **Form Usability:** Integrated Phase 1 `FormFieldComponent` and `TextInputComponent` with `forId` , `aria-describedby` , `autocomplete` , and visual error messages.
- **Micro-Interactions:** Loading states on buttons, clear feedback alerts with `role="alert"` and `aria-live="polite"` .

H. Verification & Quality Gates

1. TypeScript Strict Type Check

```
npx tsc --noEmit
# Exit code: 0
```

2. ESLint Check

```
npm run lint
# Linting "al-azhari-frontend"...
# All files pass linting.
# Exit code: 0 (0 errors, 0 warnings)
```

3. Stylelint Check

```
npm run lint:style
# stylelint "src/**/*.scss"
# Exit code: 0 (0 errors, 0 warnings)
```

4. Angular Unit Test Suite

```
npm test -- --watch=false --no-progress
# Chrome Headless 153.0.0.0 (Linux 0.0.0): Executed 166 of 166 SUCCESS (1.465 secs / 1.294 secs)
# TOTAL: 166 SUCCESS (0 FAILED)
# Exit code: 0
```

- **Phase 0 Baseline Tests:** 13 PASS
- **Phase 1 Primitives & Stylebook Tests:** 56 PASS
- **Phase 2 HTTP Infrastructure Tests:** 46 PASS
- **Phase 3 Authentication Tests:** 51 PASS
- **Total Passing Tests:** 166 / 166 PASS

5. Angular Production Build

```
npm run build
# Application bundle generation complete. [4.012 seconds]
# Output location: frontend/dist/al-azhari-frontend
# Exit code: 0 (0 warnings, 0 errors)
```

I. Strict Boundary Check

- **No Phase 4+ implementation was performed:**
 - Customer profile, customer addresses, and customer dashboard: **NOT IMPLEMENTED**
 - Catalog, categories, products, and product details: **NOT IMPLEMENTED**
 - Cart, inventory checkout, and orders: **NOT IMPLEMENTED**
 - Admin management features and store services: **NOT IMPLEMENTED**
- **Backend Codebase:**
 - Exactly 0 modifications to Back-end/ .

PHASE 3 COMPLETE. Execution stopped intentionally at Phase 3. No Phase 4+ implementation was performed.